



TALK-TO-DATA (T2D)

Phase 5 Annexes: Prototype / MVP build

Daniel Brulé

[LinkedIn](#) - [GitHub](#)

Version 1.0 - June 2026

Table of Contents

1.	How to use this annex	4
2.	Activity 1: Confirm MVP scope and build plan.....	5
2.1.	MVP scope confirmation template	5
2.2.	Example: included and excluded question list	5
2.3.	MVP technology stack checklist	6
2.4.	Build backlog template	7
2.5.	Acceptance criteria checklist.....	8
2.6.	Prototype simplification log.....	9
2.7.	Dependency and blocker log	9
2.8.	Alignment meeting agenda.....	10
3.	Activity 2: Confirm MVP governance, ownership and decision rights.....	11
3.1.	MVP governance register	11
3.1.	Decision and approval route	12
3.1.	Change log	13
3.1.	Risk acceptance log.....	13
3.1.	Example: MVP governance register entry	13
3.1.	Governance meeting agenda.....	14
4.	Activity 3: Set up environment, CI/CD and test harness	15
4.1.	MVP environment setup checklist	15
4.2.	Version-control and deployment checklist.....	15
4.3.	Delivery tracking setup.....	16
4.4.	Example tools that may be used	16
4.5.	Test harness checklist.....	17
4.6.	Logging and tracing checklist	17
4.7.	Cost and latency capture template.....	18
4.8.	Example: lightweight MVP environment pattern	18
5.	Activity 4: Connect approved data, metadata and tools	19
5.1.	Approved connection register.....	19
5.2.	Access boundary checklist	19
5.3.	Access enforcement pattern register	19
5.4.	Metadata availability checklist	20
5.5.	Connection test template	20
5.6.	Example: regional sales access enforcement	21

5.7.	Tool boundary checklist	21
6.	5. Activity 5: Implement metadata retrieval and grounding	22
6.1.	Retrieval configuration checklist	22
6.2.	Grounding rules template	22
6.3.	Retrieval test set template	23
6.4.	Metadata gap log	23
6.5.	Example: retrieval trace for a supported question	23
6.6.	Retrieval quality summary	24
7.	Activity 6: Implement AI/model routing and clarification	25
7.1.	Example: model/task selection options based on June 2026 model landscape	25
7.2.	Routing rules template	26
7.3.	Clarification trigger checklist	27
7.4.	Routing and clarification test template	27
7.5.	Model comparison log	27
7.6.	Example: routing decision record	28
8.	Activity 7: Implement query generation, validation and execution	29
8.1.	Query generation configuration checklist	29
8.2.	Rule-based validation checklist	29
8.3.	Execution guardrail register	30
8.4.	Query test case template	30
8.5.	Query and execution evidence log	31
8.6.	Example: blocked unsafe query	31
8.7.	Example: access-constrained query	31
8.8.	Query performance summary	32
9.	Activity 8: Implement result interpretation and answer generation	33
9.1.	Answer format rules template	33
9.2.	Evidence boundary rules	33
9.3.	How to test answer overreach	34
9.4.	Answer-quality review template	35
9.5.	Answer issue log	35
9.6.	Example: safe answer when causal evidence is missing	36
9.7.	Example: answer format for a supported metric question	36
9.8.	Empty, partial and failed result handling	36
10.	Activity 9: Build lightweight UI and feedback capture	37

10.1.	UI option checklist.....	37
10.2.	Example: technology options for lightweight UI	37
10.3.	Answer display checklist.....	38
10.4.	Feedback capture template	38
10.5.	UI security checklist	38
10.6.	Example: lightweight web UI layout	39
10.7.	Example: feedback-to-trace link.....	39
11.	Activity 10: Tune, test and package evidence	40
11.1.	Tuning workflow.....	40
11.2.	Concrete tuning levers	40
11.3.	Model tuning and routing options	41
11.4.	Prompt tuning checklist	41
11.5.	Tuning record template	42
11.6.	Regression check template	42
11.7.	Issue classification and triage	42
11.8.	Phase 6 evidence pack checklist	43
11.9.	Example: controlled prompt tuning record	43
11.10.	Example: model routing tuning record	44
11.1.	10.5. Versioning approach for MVP tuning	44

1. How to use this annex

These annexes provide optional working templates for Phase 5. They are not all mandatory outputs. Teams should select the artefacts that match the delivery intent, risk level and maturity of the MVP. For a POC, several templates may be simplified. For an MVP intended for validation and user testing, the evidence, governance, access, query-control and tuning records should be treated more seriously.

2. Activity 1: Confirm MVP scope and build plan

2.1. MVP scope confirmation template

Item	Description
Delivery intent	POC / MVP / pilot preparation / production path
MVP objective	What the build is intended to prove or enable
Target users	User group included in the MVP
Business domain	Domain covered by the MVP
Priority questions	Questions included in the current build
Supported answer types	Number, table, chart, narrative, caveated answer, clarification, refusal
Main data assets	Approved Phase 3 assets used by the MVP
Metadata sources	Metric cards, semantic register, examples, caveats, data catalogue, other
AI/model components	Intent classification, retrieval ranking, SQL generation, answer generation, follow-up handling
Tool interfaces	Metadata retrieval, query validation, SQL execution, logging, feedback capture
User interface	Chat shell, embedded interface, notebook, internal app, BI integration, other
Main exclusions	Questions, users, datasets, tools, answer types or product features out of scope
Known caveats	Limitations that must be visible during testing or validation
Evidence needed for Phase 6	Tests, logs, traces, examples, issue records, quality findings
Decision owner	Person or group approving the MVP scope
Build owner	Person or group accountable for delivery
Last updated	Date of latest scope confirmation

2.2. Example: included and excluded question list

Priority question	Included ?	Reason	Required assets	Required AI/model steps	Caveat / limitation	Owner
Revenue by region last month	Yes	Core MVP question	Revenue mart, region hierarchy, net revenue metric card	Retrieval, SQL generation, answer generation	Current month may be provisional	Finance analytics
Margin by product	No / Deferred	Cost allocation unresolved	Revenue mart, product hierarchy, cost model	Retrieval, SQL generation	Margin definition not approved	Commercial finance

Top customer s by declining spend	Include with caveat	Useful for user testing	Revenue mart, customer master	Retrieval, SQL generation, interpretation	Customer hierarchy may affect ranking	Sales operations
--	---------------------	-------------------------	-------------------------------	---	---------------------------------------	------------------

Suggested status values:

Status	Meaning
Include	Ready to build into the MVP.
Include with caveat	Usable for MVP testing, but limitation must be shown or controlled.
Remediate	Valuable, but requires data, logic, access or design work first.
Defer	Potentially useful later, but not needed for the current MVP.
Exclude	Not suitable for this scope or delivery stage.

2.3. MVP technology stack checklist

Area	Decision to confirm	Selected option	Temporary substitution?	Owner
Application framework	How the MVP application or service will be built			
User interface	How users/testers will interact with the MVP			
Orchestration layer	How the question-to-answer flow is implemented			
Model provider / gateway	How model access, routing and credentials are managed			
Embedding / retrieval store	Where metadata embeddings or search indexes are stored			
Metadata source	Where metric, dimension, join, caveat and example metadata is held			
Query validation	How generated queries are checked before execution			
Query execution route	Which warehouse, semantic layer, API or query engine is used			
Access control	Where user permissions and data restrictions are enforced			
Logging and tracing	Where prompts, tool calls, queries, errors and traces are captured			
Cost tracking	How model, query and infrastructure cost are estimated or tracked			
Latency tracking	How response time is captured by step or interaction			
CI/CD	How code, prompts, configuration and deployment are managed			

Hosting environment	Where the MVP runs: local, cloud dev, internal platform, managed service			
Secrets management	How credentials and keys are stored and rotated			
Monitoring dashboard	How early quality, error, cost and latency signals are viewed			

Practical note: temporary substitutions are acceptable for a POC or MVP, but they should be labelled clearly. A temporary spreadsheet, local vector store, manual approval route or simplified UI should not be mistaken for the future production design.

2.4. Build backlog template

Backlog item	Area	Description	Priority	Owner	Dependency	Acceptance evidence
Build metadata retrieval	Metadata / retrieval	Retrieve metric cards and relevant examples for each supported question	High	AI engineer	Approved metadata source	Retrieval test results
Implement SQL generation prompt	AI/model	Generate bounded SQL from approved context	High	AI engineer	Queryable assets, metric cards	Golden question SQL examples
Add validation gate	Validation	Block unsafe tables, unsupported joins, missing filters or high-cost queries	High	Data engineer / AI architect	Approved validation rules	Validation pass/fail logs
Build answer shell	UI	Show answer, caveats, source and feedback option	Medium	Software engineer	Answer format	UI test
Add cost and latency capture	Observability	Capture model calls, query execution time and total response time	Medium	Software engineer	Logging setup	Cost/latency dashboard

Suggested backlog areas:

Area	Example items
Data connection	Warehouse access, approved views, query execution route
Metadata	Metric cards, semantic registry, examples, caveats, embeddings
AI/model	Prompts, routing, model comparison, answer generation
Validation	SQL validation, permission checks, row limits, query cost checks
Security	Secrets, access controls, logging exclusions, data exposure checks
UI	Chat shell, answer display, source visibility, feedback capture
Observability	Logs, traces, cost, latency, error capture
Testing	Golden questions, failure cases, regression checks
Governance	Decision log, change route, issue ownership
Handover	Evidence pack, backlog, production-readiness gaps

2.5. Acceptance criteria checklist

Area	Acceptance question	Status	Evidence
Scope	Are users, questions, answer types and exclusions confirmed?		
Data	Are approved data assets accessible in the MVP environment?		
Metadata	Are required metric, dimension, join, filter and caveat records available?		
AI/model use	Are AI tasks, model choices and deterministic controls documented?		
Query generation	Can supported questions generate bounded queries?		
Validation	Are unsafe, invalid or unsupported queries blocked or routed?		
Answer generation	Do answers show sources, assumptions, caveats and limitations?		
Interface	Can testers use the MVP flow without manual intervention?		
Logging	Are prompts, retrieval, queries, validation results, errors and answers traceable?		
Cost / latency	Are cost and response-time signals captured at least directionally?		
Security	Are basic access, secrets and exposure controls in place?		
Testing	Are representative success and failure cases available?		
Governance	Are owners, decision rights, change route and issue route clear?		
Handover	Is the evidence pack sufficient for Phase 6 validation planning?		

Suggested status values: Not started / In progress / Ready / Ready with caveat / Blocked / Not applicable.

2.6. Prototype simplification log

Simplification	Why accepted	Risk	Valid for POC only ?	Blocks validation ?	Owner	Next action
Metadata held in spreadsheet	Faster MVP setup	Weak version control and automation	Yes	Maybe	Data / semantic owner	Move to versioned file or metadata table
Limited user authentication	Internal test only	Does not prove final access model	Yes	Yes for pilot	Security lead	Confirm target auth route
Manual review of failed SQL	Speeds early iteration	Not scalable	No	No if logged	Evaluation owner	Define automated validation path
Single model for all tasks	Simpler implementation	Cost and latency may be higher	No	Maybe	AI architect	Compare models by task
Lightweight UI	Enough for testing flow	Does not prove product UX	No	No	Product owner	Capture UX gaps for pilot

The simplification log should distinguish acceptable MVP shortcuts from issues that must be fixed before validation, pilot or production. The main risk is not taking shortcuts; it is forgetting that they were shortcuts.

2.7. Dependency and blocker log

Dependency / blocker	Area	Impact	Owner	Required decision or action	Target date	Status
Access to revenue mart not granted	Data access	Blocks core question testing	Data owner	Approve dev/test access		
Metric cards incomplete	Metadata	Weak grounding and caveat handling	Semantic owner	Complete required metric fields		
Model provider not approved	AI / security	Blocks model integration	Security lead	Confirm approved model route		

Logging destination unclear	Observability	Weak traceability	Platform owner	Confirm log store and retention		
Query cost threshold not agreed	Cost control	Risk of expensive execution	Product / data owner	Define MVP cost threshold		

2.8. Alignment meeting agenda

Purpose: confirm that the MVP scope, technology choices, exclusions, ownership and evidence expectations are understood before implementation starts.

Suggested participants: product owner, AI / solution architect, AI / ML engineer, data / semantic owner, analytics / data engineer, software engineer, security / governance lead, evaluation owner and operating owner.

Agenda item	Decision to confirm
MVP intent	POC learning, controlled MVP, pilot preparation or production path
Supported users and questions	What is included and excluded
Technology stack	What will be used for MVP build and what is temporary
AI/model use	Which AI tasks and model patterns are included
Data and metadata dependencies	Which assets and metadata sources must be ready
Validation and evidence	What must be shown before Phase 6
Security and access assumptions	What controls are in place and what remains unresolved
Cost and latency capture	What signals will be captured during the MVP
Governance and ownership	Who approves changes, accepts risks and owns defects
Backlog creation	How product and engineering will shape delivery priorities

Expected meeting outputs:

Output	Owner
Confirmed MVP scope	Product owner
Confirmed technology stack	AI / solution architect
Confirmed exclusions and simplifications	Product owner / AI architect
Confirmed evidence expectations	Evaluation owner
Confirmed issue and decision route	Product owner / operating owner
Initial build backlog	Product owner and engineering lead

3. Activity 2: Confirm MVP governance, ownership and decision rights

3.1. MVP governance register

Purpose: create a single register showing who owns, validates, approves and accepts key MVP build decisions.

Area	Decision / responsibility to capture	Owner	Validator / approver	Evidence / record	Status
Scope	Who owns MVP scope, exclusions and changes to supported questions?			Scope statement / decision log	
Data access	Who owns access to approved data assets and dev/test permissions?			Access approval / access log	
Metadata	Who owns metric, dimension, join, caveat and example metadata?			Metadata register / version history	
Retrieval	Who owns retrieval quality and grounding behaviour?			Retrieval tests / failure log	
AI/model routing	Who owns model selection, routing logic and task allocation?			Model decision record	
Prompts	Who owns prompt changes and prompt versioning?			Prompt registry / change log	
Query generation	Who owns generated-query behaviour?			Query test results	
Query validation	Who owns validation gates and blocking rules?			Validation rules / test results	
Query execution	Who owns execution route, limits and query cost assumptions?			Execution logs / cost checks	
Answer generation	Who owns answer format, caveats and safe-failure behaviour?			Answer examples / review notes	
Security and exposure	Who owns access assumptions, sensitive fields, logging exclusions and exposure risks?			Security review notes	
Logging and tracing	Who owns trace capture, retention assumptions and diagnostic evidence?			Trace logs / observability notes	
Testing evidence	Who owns test cases, results, defects and evidence quality?			Test pack / issue log	

Phase 6 handover	Who owns the handover to formal validation?			Handover pack	
Production-readiness gaps	Who owns unresolved gaps and next actions?			Gap list / backlog	

Suggested status values: Not assigned / Assigned / Confirmed / In review / Blocked / Not applicable.

3.1. Decision and approval route

Change type	Example	Approval owner	Evidence required	Must be approved before change?
MVP scope change	Add a new question or answer type	Product owner	Impact on scope, tests, access and timeline	Yes
Data asset change	Use a different table, view or semantic model	Data owner / semantic owner	Source approval and caveat review	Yes
Metadata change	Add or revise metric, dimension, join or caveat metadata	Semantic owner	Version update and review notes	Yes
Prompt change	Change SQL-generation or answer-generation prompt	AI / solution architect	Prompt version, reason and regression result	Yes for MVP
Model change	Switch model used for SQL generation or answer generation	AI / solution architect	Model comparison, cost/latency impact and test result	Yes
Routing change	Change which model/tool handles a task	AI / solution architect	Routing decision and test result	Yes
Validation change	Add, remove or relax a validation rule	Evaluation owner / governance lead	Failure analysis and test evidence	Yes
Tool change	Add or change metadata, validation or execution tool	AI architect / security lead	Permission review and logging design	Yes
Access assumption change	Change user group, access route or masking assumption	Security / governance lead	Access review and risk decision	Yes
Logging change	Change prompt, query, result or trace retention	Security / operating owner	Retention and exposure review	Yes

Accepted limitation	Allow a known limitation into Phase 6 validation	Product owner / evaluation owner	Risk note, owner and next action	Yes
----------------------------	--	----------------------------------	----------------------------------	-----

3.1. Change log

Change ID	Date	Change type	Description	Reason	Owner	Approved by	Impact	Evidence link / reference
-----------	------	-------------	-------------	--------	-------	-------------	--------	---------------------------

Suggested change types: Scope / Metadata / Prompt / Model / Routing / Validation / Tool / Access / Logging / UI / Test / Handover.

3.1. Risk acceptance log

Risk / limitation	Area	Why accepted for MVP	Impact if wrong	Valid for POC only?	Blocks Phase 6?	Blocks pilot?	Owner	Next action
-------------------	------	----------------------	-----------------	---------------------	-----------------	---------------	-------	-------------

Suggested decision values:

Decision	Meaning
Accept for POC	Acceptable only for learning or technical proof.
Accept for MVP validation	Acceptable for formal validation if documented and tested.
Accept for pilot with control	Acceptable for pilot only with caveat, monitoring or restricted use.
Must remediate	Must be fixed before the next phase.
Defer	Not required for current scope, but should remain in backlog.

3.1. Example: MVP governance register entry

This is an illustrative example only. It shows the level of detail expected, not a required format.

Area	Decision / responsibility to capture	Owner	Validator / approver	Evidence / record	Status
Prompt changes	SQL-generation prompt changes must be versioned and regression-tested before use in MVP validation	AI / solution architect	Evaluation owner	Prompt registry, regression result, change log	Confirmed
Metadata	Net revenue metric card and regional hierarchy must be approved before supported revenue questions are tested	Semantic owner	Finance SME	Metric card version, approval note	In review
Query validation	Queries must be blocked if they use non-approved tables, missing mandatory filters or exceed cost threshold	Data engineer	Security / governance lead	Validation rule list, failed-query examples	Confirmed

Phase 6 handover	Evidence pack must include test cases, traces, answer examples, failed tests and unresolved limitations	Evaluation owner	Product owner	Evidence pack checklist	Assigned
-------------------------	---	------------------	---------------	-------------------------	----------

3.1. Governance meeting agenda

Purpose: confirm ownership, decision rights, validation responsibilities and issue routes before implementation decisions start accumulating.

Suggested participants: product owner, AI / solution architect, AI / ML engineer, data / semantic owner, analytics / data engineer, software engineer, security / governance lead, evaluation owner, operating owner and relevant business SME.

Agenda item	Decision to confirm
MVP governance register	Where ownership, decisions, risks and changes will be recorded
Component ownership	Who owns each build component and evidence area
Validation responsibilities	Who validates generated queries, answers, caveats, refusals, logs and tests
Approval route	Who approves material changes to scope, prompts, models, tools, metadata, validation or access
Issue and risk route	How defects, failed tests, limitations and production-readiness gaps are logged and escalated
Change log	What changes must be recorded during MVP build
Risk acceptance	Who can accept limitations for MVP validation or pilot preparation
Phase 6 handover	Who owns evidence quality and validation readiness
Production governance gaps	Which governance gaps are acceptable for MVP and which must be closed later

Expected meeting outputs:

Output	Owner
Confirmed MVP governance register owner	Product owner / operating owner
Confirmed component ownership map	AI / solution architect
Confirmed validation responsibility map	Evaluation owner
Confirmed approval route	Product owner / governance lead
Confirmed issue and risk route	Operating owner
Confirmed Phase 6 handover owner	Evaluation owner
Initial production governance gap list	Operating owner / governance lead

4. Activity 3: Set up environment, CI/CD and test harness

4.1. MVP environment setup checklist

Area	Decision / setup item	Owner	Status	Notes
Development environment	Where developers build and run the MVP locally or in isolated dev space			
Test environment	Where the MVP is tested against approved data, metadata and tools			
Environment creation	Script, container, infrastructure template or documented setup route			
Approved data access	Access to scoped Phase 3 assets only			
Metadata access	Access to approved metric, dimension, join, caveat and example metadata			
Model access	Approved model provider, gateway or private endpoint			
Tool access	Metadata retrieval, query validation, query execution, logging and feedback tools			
Secrets handling	Credentials, API keys and service accounts managed safely			
Environment variables	Required configuration documented and separated by environment			
Test data limits	Any row, query, user, domain or cost limits for the MVP environment			

Practical note: for a POC, this may be lightweight. For an MVP, developers should be able to create a small local or isolated environment through a documented script, container or infrastructure template so they can work independently and reproduce issues.

4.2. Version-control and deployment checklist

Area	What should be versioned or controlled	Required for MVP?	Notes
Application code	UI, APIs, orchestration logic and integration code	Yes	
Prompts	System prompts, task prompts, SQL-generation prompts and answer-generation prompts	Yes	
Model configuration	Model names, routing rules, temperature/settings and fallback logic	Yes	
Validation rules	SQL validation, access checks, row limits, cost limits and blocked actions	Yes	
Metadata artefacts	Deployable metric cards, examples, caveats, mappings or retrieval configuration	Yes, if used at runtime	
Test cases	Success cases, failure cases, unsafe requests and regression tests	Yes	

Environment configuration	Non-secret environment configuration and deployment settings	Yes	
Secrets	References only; secrets should not be stored in the repository	Yes	
Documentation	Setup instructions, runbook notes and known limitations	Recommended	

Suggested tooling examples: Git, GitHub, GitLab, Azure DevOps, Bitbucket, GitHub Actions, GitLab CI, Azure Pipelines, Jenkins or equivalent. The specific tool matters less than having a controlled way to version, review, test and deploy changes.

4.3. Delivery tracking setup

Tracking area	What to capture	Example tool route
Build backlog	MVP implementation work by component	Jira, Azure DevOps, GitHub Issues
Defects	Failed tests, broken flows, incorrect answers, validation failures	Jira, Azure DevOps, GitHub Issues
Prompt / model issues	Model misrouting, weak prompts, inconsistent answers, hallucinated context	Issue board or experiment tracker
Data / metadata issues	Missing metric cards, weak caveats, wrong retrieval, unavailable assets	Issue board linked to owner
Security / access issues	Permission gaps, overexposure, secret handling, logging exclusions	Security backlog or risk log
Cost / latency issues	Expensive model calls, slow queries, high response time	Issue board linked to observability
Production-readiness gaps	Items not blocking MVP but required before pilot or production	Hardening backlog

4.4. Example tools that may be used

This is an illustrative list only. The tools should be selected based on the organisation's existing platform, security requirements, team skills and delivery stage.

Area	Example tools
Version control	GitHub, GitLab, Azure Repos, Bitbucket
Delivery tracking	Jira, Azure DevOps Boards, GitHub Issues, Linear
CI/CD	GitHub Actions, GitLab CI, Azure Pipelines, Jenkins
Local development	Docker, Docker Compose, Dev Containers, Makefiles, Poetry, uv
Infrastructure as code	Terraform, Pulumi, Bicep, CloudFormation
Secrets management	Azure Key Vault, AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault
Model gateway / routing	LiteLLM, Portkey, provider-native gateways, internal model gateway
Experiment and prompt tracking	MLflow, Weights & Biases, LangSmith, PromptLayer, Helicone
Observability and tracing	OpenTelemetry, Datadog, Grafana, Prometheus, Elastic, Splunk
Application logging	CloudWatch, Azure Monitor, Google Cloud Logging, ELK stack
Testing	pytest, dbt tests, Great Expectations, Soda, SQL unit tests

Data warehouse / query engine	Snowflake, BigQuery, Databricks SQL, Redshift, Synapse, PostgreSQL
Metadata / catalogue	DataHub, OpenMetadata, Collibra, Alation, Microsoft Purview
Dashboarding	Power BI, Tableau, Looker, Grafana, Metabase

4.5. Test harness checklist

Test type	Purpose	Example evidence
Supported question test	Confirms approved questions can run end-to-end	Question, trace ID, answer, generated query
Metadata retrieval test	Confirms the right metric, dimension, join, caveat or example is retrieved	Retrieved artefacts and ranking result
SQL generation test	Confirms generated query uses approved assets, filters and grain	Generated SQL and expected SQL comparison
Validation failure test	Confirms unsafe, unsupported or excessive queries are blocked	Failed validation result
Access test	Confirms user or role restrictions are respected	Access result and blocked examples
Answer-quality test	Confirms answer is clear, caveated and grounded in results	Answer examples and reviewer notes
Safe-failure test	Confirms clarification, refusal or escalation works when needed	Clarification/refusal examples
Regression test	Confirms previous working cases still pass after changes	Test run results
Cost / latency test	Captures response time and major cost drivers	Cost/latency summary

4.6. Logging and tracing checklist

Step to trace	What to capture	Notes
User request	Question, timestamp, session ID, user role or test persona	Avoid unnecessary personal data.
Intent / routing	Detected intent, route selected, model or rule used	Useful for debugging misclassification.
Metadata retrieval	Search query, retrieved artefacts, versions and ranking	Critical for grounding failures.
Prompt / model call	Prompt version, model used, settings and response	Apply retention and redaction rules.
Generated query	SQL or query payload before validation	Required for evaluation and debugging.
Validation	Rules applied, pass/fail result and reason	Required before execution.
Execution	Query status, runtime, row count, cost estimate where available	Useful for performance and cost analysis.
Answer generation	Result summary, caveats used and final answer	Required for answer-quality review.
Error / refusal	Error, refusal reason or escalation route	Important for safe-failure analysis.
Feedback	Tester feedback, correction or issue link	May be lightweight before pilot.

4.7. Cost and latency capture template

Interacti on ID	Questi on type	Mod el calls	Retriev al time	Query executi on time	Answer generati on time	Total laten cy	Query cost estima te	Model cost estima te	Note s
--------------------	-------------------	--------------------	--------------------	-----------------------------	-------------------------------	----------------------	-------------------------------	-------------------------------	-----------

Suggested summary views:

View	Why useful
Average latency by question type	Shows whether the flow is usable for realistic testing.
Slowest interactions	Helps identify retrieval, model or query bottlenecks.
Model calls per interaction	Shows whether orchestration is becoming too expensive or complex.
Query execution cost by question	Highlights expensive queries before pilot.
Failure rate by step	Shows whether problems are in retrieval, generation, validation, execution or answers.

4.8. Example: lightweight MVP environment pattern

This is an illustrative example only. It shows a possible MVP setup, not a required architecture.

Layer	Example setup
Repository	Git repository with application code, prompts, validation rules and tests
Development setup	Docker Compose or setup script for local app, mock services and test configuration
Test environment	Cloud dev/test deployment connected to approved warehouse views and metadata store
Metadata store	Versioned YAML/JSON or metadata table containing metric cards and caveats
Retrieval	Simple vector index or search service over approved metadata
Model access	Approved model gateway or provider endpoint with separate test credentials
Query execution	Read-only warehouse role with scoped access and row/query limits
Logging	Structured logs with trace ID across retrieval, model calls, validation and execution
Delivery tracking	Jira, Azure DevOps or GitHub Issues for build items, defects and production gaps
Dashboard	Basic view of errors, latency, query cost, model calls and test pass/fail status

5. Activity 4: Connect approved data, metadata and tools

5.1. Approved connection register

Connection	Type	Purpose	Owner	Access method	Environment	Status	Notes
	Data asset / metadata source / tool / API				Dev / test / MVP		

Suggested connection types:

Type	Example
Data asset	Approved table, view, mart, semantic-layer object
Metadata source	Metric cards, semantic register, caveat register, example query store
Retrieval service	Search index, vector store, catalogue API
Query validation tool	SQL parser, allow-list checker, dry-run service, policy checker
Query execution route	Warehouse role, semantic-layer API, governed metric API
Logging / tracing	Trace store, application logs, model-call logs
Feedback capture	Issue form, feedback table, lightweight UI capture

5.2. Access boundary checklist

Boundary area	Question to confirm	Status	Evidence
Approved assets	Are only approved Phase 3 assets connected?		
Blocked assets	Are excluded tables, fields, tools or actions blocked?		
User entitlement source	Where are user roles, security levels or permitted dimensions retrieved from?		
Row-level control	Are row-level restrictions enforced before execution?		
Column-level control	Are restricted columns hidden, masked or blocked?		
Masking	Are sensitive values masked before results reach the model or answer layer?		
Aggregation limits	Are small cohorts, restricted drill-downs or inference risks controlled?		
Tool permissions	Can tools perform only approved actions?		
Logging exclusions	Are sensitive prompts, results or fields excluded or redacted where needed?		
Failure behaviour	What happens when access is denied or metadata is missing?		

5.3. Access enforcement pattern register

Pattern	Where access is enforced	Strength	Watch-out
---------	--------------------------	----------	-----------

Warehouse row-level security	Data warehouse policy applies user or role restrictions	Strong	Requires correct identity propagation or role mapping
Semantic-layer policy	Approved semantic layer applies row, column or metric restrictions	Strong	Depends on semantic-layer maturity and coverage
Authorised API	API exposes only approved data or metrics for the user context	Strong	Less flexible for exploratory analysis
Controlled query rewrite	Policy layer injects approved filters before execution	Acceptable if governed	Must be tested, logged and not model-controlled
Answer-layer filtering only	System queries broad data and hides restricted results in the response	Weak	Avoid except for non-sensitive POC learning

5.4. Metadata availability checklist

Metadata item	Required for MVP?	Source	Runtime usable?	Owner	Gap / action
Metric definitions					
Dimensions					
Join rules					
Standard filters					
Caveats					
Example questions					
Example SQL					
Access rules					
User entitlement metadata					
Data freshness / latency					
Source trust status					

5.5. Connection test template

Test question	Data asset used	Metadata retrieved	Tool path used	Access control applied	Result	Issue / action
			Retrieval / validation / execution / logging		Pass / fail / partial	

Test guidance:

Test type	Purpose
Supported question test	Confirms the connection path works for an approved MVP question.
Access-restricted question test	Confirms user entitlements restrict rows, columns or metrics before execution.
Blocked asset test	Confirms excluded assets or fields cannot be queried.

Metadata gap test	Confirms the system clarifies, refuses or escalates when metadata is missing.
Tool failure test	Confirms the system handles unavailable tools safely.

5.6. Example: regional sales access enforcement

This is an illustrative example only. It shows the access-control logic, not a required implementation.

Scenario: a regional sales manager is entitled to see UK and France only.

Step	Expected behaviour
User asks	“Show me sales by region last month.”
Entitlement lookup	System retrieves the user’s permitted regions from governed security metadata or an access service.
Policy enforcement	Query, semantic-layer request or API call is constrained to UK and France before execution.
Execution	Only permitted rows or aggregates are returned.
Answer generation	The model receives only permitted results and explains the answer with any relevant caveats.
Logging	Trace records the access policy applied, without exposing unnecessary restricted data.

Example constrained query:

```
SELECT region, SUM(sales_amount) AS sales
FROM approved_sales_region_mart
WHERE region IN ('UK', 'France')
      AND sales_month = DATE '2026-05-01'
GROUP BY region;
```

The model should not decide which regions the user can access. User entitlement should come from a trusted identity, policy or governed metadata source and be enforced before restricted results reach the model or answer layer.

5.7. Tool boundary checklist

Tool	Allowed actions	Blocked actions	Required logs	Owner
Metadata retrieval tool	Search approved metadata and examples	Retrieve unapproved or deprecated context	Search query, retrieved artefacts, versions	
Query validation tool	Validate syntax, scope, tables, joins, filters and limits	Approve access by itself	Query, checks applied, pass/fail reason	
Query execution tool	Execute read-only approved queries	Write, update, delete, unrestricted SQL, unsupported assets	Query ID, status, duration, row count, cost signal	
Logging / tracing tool	Capture required diagnostic events	Store unnecessary sensitive data	Trace ID, event type, retention status	
Feedback / issue tool	Capture tester feedback and defects	Expose restricted data in issue text	Issue ID, owner, severity	

6. 5. Activity 5: Implement metadata retrieval and grounding

6.1. Retrieval configuration checklist

Area	Decision / configuration	Owner	Status	Notes
Metadata sources	Which approved metadata sources are included in retrieval?			
Retrieval method	Keyword search, semantic search, hybrid search, catalogue/API lookup or rules			
Ranking logic	How retrieved artefacts are ranked and selected			
Scope filters	Domain, user group, approved question set or delivery-stage filters			
Version filters	Draft, approved, deprecated or retired metadata handling			
Access filters	How user entitlement affects retrievable metadata			
Context limit	Maximum number of artefacts, examples, columns or tokens passed forward			
Prompt packaging	How retrieved metadata is structured for the model or tools			
Fallback behaviour	What happens when retrieval is weak, missing or conflicting			
Logging	What retrieval inputs, outputs, versions and scores are captured			

6.2. Grounding rules template

Rule area	Rule to define	Example decision
Approved artefacts	Which artefacts can be used for grounding?	Use only approved metric cards and current semantic records.
Metadata priority	Which source wins when metadata conflicts?	Certified semantic registry overrides dashboard notes.
Business terms	How synonyms and ambiguous terms are handled	“Revenue” defaults to net revenue only for approved finance questions.
Metric selection	How the correct metric is selected	Retrieve metric card by domain, synonym and supported question.
Dimension selection	Which dimensions can be used for filtering or grouping	Use only dimensions listed as approved for the selected metric.
Join selection	How join paths are constrained	Use only approved joins from the join register.
Caveat handling	Which caveats must be passed into answer generation	Mandatory caveats are always included in answer context.
Access metadata	How user entitlement limits metadata or examples	Do not retrieve metadata for assets the user cannot query.
Examples	How example questions or SQL are used	Examples guide structure but cannot override approved rules.
Missing context	What happens when required metadata is unavailable	Clarify, refuse or route to issue backlog.

6.3. Retrieval test set template

Test question	Expected metadata	Expected behaviour	Result	Issue / action
	Metric, dimension, join, caveat, example, access rule	Retrieve / clarify / refuse / escalate	Pass / fail / partial	

Suggested test types:

Test type	Purpose
Supported question	Confirms the right metadata is retrieved for an approved MVP question.
Synonym question	Confirms business terms map to approved metrics or dimensions.
Ambiguous question	Confirms the system clarifies rather than guessing.
Out-of-scope question	Confirms unsupported domains or metrics are refused or routed.
Access-restricted question	Confirms user entitlements restrict metadata and examples.
Conflicting metadata	Confirms priority rules are applied when sources disagree.
Missing metadata	Confirms the system fails safely rather than inventing context.

6.4. Metadata gap log

Gap	Affected question	Gap type	Impact	Owner	Decision	Next action
		Missing / stale / conflicting / inaccessible / not runtime-usable			Fix / caveat / defer / exclude	

Suggested gap types:

Gap type	Meaning
Missing	Required metadata does not exist.
Stale	Metadata exists but is outdated or not current.
Conflicting	Multiple sources define the term or rule differently.
Inaccessible	Metadata exists but cannot be retrieved by the MVP.
Not runtime-usable	Metadata is documented but not structured enough for retrieval.
Not permitted	Metadata should not be exposed for the user or scope.

6.5. Example: retrieval trace for a supported question

This is an illustrative example only. It shows the level of evidence expected, not a required format.

Trace item	Example value
User question	"What was revenue by region last month?"
Detected domain	Finance / commercial performance
Retrieved metric	Net revenue metric card v1.2
Retrieved dimensions	Region hierarchy v1.0, month period definition v1.1
Retrieved source	approved_sales_revenue_mart
Retrieved joins	No join required for MVP question
Retrieved caveats	Current month provisional, excludes cancelled invoices
Retrieved examples	Example SQL: revenue by region by month

Access context	User entitled to UK and France regions
Grounding decision	Proceed to query generation with mandatory caveats
Retrieval status	Pass
Logged evidence	Trace ID, retrieved artefact IDs, versions and ranking scores

6.6. Retrieval quality summary

Measure	What it indicates	MVP target / observation
Expected artefact retrieved	Whether the right metric, dimension or caveat was found	
Wrong artefact retrieved	Whether retrieval selects irrelevant or unsafe context	
Missing artefact	Whether required metadata is unavailable	
Excess context	Whether too much context is passed to the model	
Clarification triggered	Whether ambiguity is handled safely	
Refusal triggered	Whether unsupported questions are blocked	
Access filtering applied	Whether user entitlement filters metadata	

Practical note: these measures do not need to become a formal evaluation framework in Phase 5. They are early signals that retrieval is good enough to enter Phase 6 validation.

7. Activity 6: Implement AI/model routing and clarification

7.1. Example: model/task selection options based on June 2026 model landscape

This table is illustrative only. Model availability, pricing and capability change quickly, so teams should confirm current enterprise availability, approved providers, data-handling terms, region support and cost before implementation.

T2D task	Typical model pattern	Example models as at June 2026	Why this pattern fits	Watch-outs
Intent classification	Rules, small model or medium model	GPT-5 mini / GPT-4o mini, Claude Haiku, Gemini Flash, Ministral 3 8B / 14B	Usually structured and low-risk if domains are bounded.	Avoid using a high-cost reasoning model for simple routing.
Metadata retrieval	Embedding model + rules / search	OpenAI text-embedding models, Gemini embeddings, bge/e5-style embeddings	Retrieval quality usually depends more on metadata quality and ranking than large reasoning models.	Test false matches and missing matches.
Metadata ranking	Small or medium model	GPT-5 mini, Claude Haiku / Sonnet, Gemini Flash, Mistral Small / Ministral	Useful for selecting the best metric card, caveat or example after retrieval.	Do not let ranking override approved metadata status.
Clarification	Medium model with controlled rules	GPT-5.1 / GPT-5.2, Claude Sonnet, Gemini Flash / Pro, Mistral Medium / Large	Needs natural language, but should follow explicit clarification triggers.	Too many clarification questions harms usability; too few creates guessing.
SQL/query generation	Medium model; stronger model for complex queries	GPT-5.1 / GPT-5.2, Claude Sonnet, Gemini Pro / Flash, Mistral Large 3	Often needs structured reasoning over metrics, filters, joins and examples.	Stronger models do not remove the need for validation.
Complex SQL / edge cases	Strong reasoning model	GPT-5.2 pro / GPT-5.5, Claude Opus, Gemini Pro, Mistral Large 3	Useful where joins, ambiguity or multi-step logic are difficult.	Higher cost and latency; should be justified by test results.

Query validation support	Deterministic checks + optional model review	SQL parser, policy engine, dbt tests, plus medium/strong model review where helpful	Validation should be rule-led, with models assisting explanation or edge-case review.	Never rely on the model alone to approve execution.
Result interpretation	Medium model	GPT-5.1 / GPT-5.2, Claude Sonnet, Gemini Flash / Pro, Mistral Large	Converts result sets into understandable business language.	Must use query result, metric definition and caveats, not invent context.
Answer generation	Medium model; stronger model for nuanced explanation	GPT-5.1 / GPT-5.2, Claude Sonnet / Opus, Gemini Pro, Mistral Large 3	Needs clear language, caveats, source visibility and safe-failure behaviour.	Stronger model only if answer quality materially improves.
Follow-up handling	Medium or strong model + explicit context rules	GPT-5.1 / GPT-5.2, Claude Sonnet / Opus, Gemini Pro	Follow-ups require context inheritance, reset rules and ambiguity handling.	High risk of scope drift and accidental filter changes.
Refusal / escalation	Policy rules + model-assisted classification	Rules / policy engine plus small or medium model	Refusal should be controlled and explainable.	Do not make policy decisions purely model-led.
Sensitive / private deployment	Private or self-hosted model where required	Mistral Large 3 / Ministral 3, Llama-family models, Qwen-family models, private cloud models	May support data residency, privacy or platform-control needs.	Higher operating burden and more evaluation required.

7.2. Routing rules template

Routing area	Rule to define	Owner	Evidence
Supported domain	Which domains can be routed into the MVP flow?		
Question type	Which question types are supported: metric lookup, comparison, trend, breakdown, follow-up, clarification?		
Model task	Which model or rule path handles each task?		
Tool path	Which tools can be called for retrieval, validation, execution or logging?		
Risk level	Which requests require stronger validation, stronger model, refusal or human review?		
Fallback path	What happens when routing confidence is low or context is missing?		

Logging	What routing decision, model choice and fallback reason must be recorded?		
----------------	---	--	--

7.3. Clarification trigger checklist

Trigger	Clarification needed?	Example clarification
Ambiguous metric	Yes	"Do you mean net revenue, gross revenue or booked revenue?"
Missing time period	Yes	"Which period should I use?"
Unsupported dimension	Yes / refuse	"Product-level breakdown is not supported in this MVP. I can answer by region."
Multiple possible sources	Yes	"Should I use the finance revenue mart or sales pipeline view?"
Missing filter	Context-dependent	"Should this include all regions you are entitled to see?"
Access conflict	Refuse / explain	"You do not have access to that level of detail."
Out-of-scope question	Refuse / route	"That question is outside the current MVP scope."
Unsafe request	Refuse	"I cannot provide that output because it may expose restricted information."

7.4. Routing and clarification test template

Test question	Expected route	Expected model / rule	Expected behaviour	Result	Issue / action
	Retrieval / SQL / clarification / refusal / escalation		Answer / clarify / refuse / escalate	Pass / fail / partial	

Suggested test types:

Test type	Purpose
Supported direct question	Confirms the normal route works for approved questions.
Ambiguous metric	Confirms clarification is triggered rather than guessing.
Missing time period	Confirms required filters are requested.
Unsupported dimension	Confirms the system refuses or redirects safely.
Access-restricted request	Confirms entitlement rules override model behaviour.
Complex query	Confirms whether a stronger model is justified.
Follow-up question	Confirms context inheritance or reset behaviour.
Out-of-scope request	Confirms unsupported requests are blocked or routed.

7.5. Model comparison log

Task	Model / rule tested	Quality result	Latency	Cost signal	Failure mode	Decision
Intent classification						Use / reject / retest
Metadata ranking						Use / reject / retest
Clarification						Use / reject / retest

SQL generation						Use / reject / retest
Answer generation						Use / reject / retest
Follow-up handling						Use / reject / retest

Decision guidance:

Decision	Meaning
Use	Good enough for MVP validation.
Use with constraint	Usable only for specific question types, domains or risk levels.
Retest	More evidence needed before selection.
Reject	Quality, cost, latency or safety is not acceptable.
Escalate	Requires architecture, security, product or evaluation decision.

7.6. Example: routing decision record

This is an illustrative example only. It shows the level of evidence expected, not a required format.

Decision item	Example decision
Task	SQL generation for approved sales-performance questions
Selected model pattern	Medium model
Example model	GPT-5.2 or Claude Sonnet
Why selected	Good balance of SQL quality, cost and latency for MVP question complexity
Deterministic controls	Approved tables only, mandatory filters, join allow-list, row limit, cost threshold
Escalation path	Use stronger model only for failed complex cases after validation review
Evidence used	25 supported questions, 8 failure cases, generated SQL review, latency and cost sample
Decision owner	AI / solution architect
Validation owner	Evaluation owner and analytics engineer
Review point	Reassess after Phase 6 formal validation

8. Activity 7: Implement query generation, validation and execution

8.1. Query generation configuration checklist

Area	Decision / configuration	Owner	Status	Notes
Query generation model	Which model or model route generates queries			
Prompt version	Which prompt version is used for query generation			
Approved assets	Which tables, views, semantic objects or APIs may be queried			
Approved metrics	Which metrics can be calculated or retrieved			
Approved dimensions	Which dimensions can be filtered or grouped			
Join rules	Which joins and grains are allowed			
Mandatory filters	Which filters must always be applied			
Access context	How user entitlements are applied before execution			
Query limits	Row limit, timeout, scan limit or aggregation limit			
Blocked actions	Write, update, delete, merge, drop, export or unrestricted query actions			
Logging	Which query, validation and execution details are retained			

8.2. Rule-based validation checklist

Validation check	Purpose	Required for MVP?	Evidence
Syntax check	Confirms query can be parsed before execution	Yes	
Read-only check	Blocks write, update, delete, merge, drop or edit actions	Yes	
Approved asset check	Confirms only approved tables, views, semantic objects or APIs are used	Yes	
Blocked field check	Prevents restricted columns or sensitive fields from being queried	Yes	
Join allow-list check	Confirms only approved joins and grains are used	Yes	
Mandatory filter check	Confirms required filters, periods or exclusions are applied	Yes	
Access entitlement check	Confirms row, column, masking or role restrictions are enforced	Yes	
Row limit check	Ensures query returns bounded results	Yes	
Timeout / runtime check	Prevents long-running or runaway queries	Yes	
Scan / cost check	Blocks queries above agreed scan or cost thresholds where available	Recommended	

Aggregation / inference check	Reduces small-cohort or restricted-drilldown exposure	Recommended	
Dry-run check	Estimates execution plan, scanned data or syntax before full execution	Recommended	

8.3. Execution guardrail register

Guardrail	Example decision	Owner	Evidence
Read-only execution	MVP execution role cannot write, update, delete, merge or drop		
Approved execution route	Queries run only through approved warehouse, semantic layer or API		
Row limit	Default maximum rows returned per query		
Timeout limit	Maximum query runtime before cancellation		
Scan / cost limit	Maximum scanned data or estimated execution cost		
Concurrency limit	Maximum concurrent executions per user or environment		
Query retry limit	Maximum retries after validation or execution failure		
Restricted joins	Cross-domain or unsupported joins blocked		
Restricted exports	Large extracts or raw-data exports blocked		
Result-shape limit	Maximum rows, columns or cells passed to answer generation		

8.4. Query test case template

Test question	Expected query behaviour	Validation result	Execution result	Issue / action
	Generate / clarify / refuse / block / escalate	Pass / fail / blocked	Success / failed / not executed	

Suggested test types:

Test type	Purpose
Supported metric question	Confirms the query uses the right metric, source, filters and grain.
Breakdown question	Confirms grouping uses approved dimensions only.
Follow-up question	Confirms inherited filters or context are applied correctly.
Access-restricted question	Confirms user entitlements are enforced before execution.
Unsupported join	Confirms unsafe joins are blocked.
Blocked field	Confirms restricted columns cannot be queried.
Missing mandatory filter	Confirms query is blocked or clarification is triggered.
High-cost query	Confirms scan, cost or timeout guardrails work.
Write/edit request	Confirms non-read-only actions are blocked.
Ambiguous question	Confirms clarification happens before query generation or execution.

8.5. Query and execution evidence log

Trace ID	Question	Generated query version	Validation outcome	Execution status	Runtime	Rows returned	Cost / scan signal	Issue link
			Pass / fail / blocked	Success / failed / not executed				

Evidence to retain:

Evidence	Why it matters
Generated query	Shows what the model attempted to execute.
Metadata versions	Shows which metric, dimension, join and caveat artefacts were used.
Validation result	Shows whether the query passed rule-based checks.
Execution status	Shows whether the query ran, failed, timed out or was blocked.
Runtime and row count	Shows performance and result size.
Cost or scan estimate	Shows whether the query pattern is viable.
Error or refusal reason	Supports debugging and validation evidence.

8.6. Example: blocked unsafe query

This is an illustrative example only. It shows the type of validation evidence expected.

Item	Example
User request	“Export all customer sales records for last year.”
Generated query issue	Query attempts raw customer-level extract outside MVP scope.
Validation checks failed	Blocked field check, result-shape limit, export restriction.
Execution status	Not executed.
System response	Refuse or redirect to an approved aggregate question.
Evidence retained	Trace ID, generated query, failed validation checks, refusal reason.
Owner	Security / governance lead and evaluation owner.

8.7. Example: access-constrained query

This is an illustrative example only. It shows access enforcement before execution.

Scenario: the user is entitled to UK and France only.

```
SELECT region, SUM(sales_amount) AS sales
FROM approved_sales_region_mart
WHERE region IN ('UK', 'France')
      AND sales_month = DATE '2026-05-01'
GROUP BY region
LIMIT 100;
```

Control	Example evidence
Entitlement source	Governed access metadata / policy service returned UK and France.
Access enforcement	Region predicate applied before execution.
Approved asset	approved_sales_region_mart is in the allow-list.

Read-only execution	Query uses select-only execution role.
Limit	<code>LIMIT 100</code> applied.
Validation result	Passed.
Execution evidence	Runtime, row count and cost signal captured.

8.8. Query performance summary

Question type	Average runtime	Slowest runtime	Average rows returned	Cost / scan signal	Main issue
Supported metric lookup					
Breakdown by dimension					
Follow-up with inherited filter					
Access-restricted query					

Suggested review questions:

Question	Why it matters
Which queries are slowest?	Identifies performance risks before validation or pilot.
Which queries scan most data?	Highlights cost and optimisation needs.
Which validations fail most often?	Shows where prompts, metadata or query rules need improvement.
Which result sets are too large?	Identifies answer-generation and exposure risks.
Which failures require clarification rather than blocking?	Improves user experience without weakening controls.

9. Activity 8: Implement result interpretation and answer generation

9.1. Answer format rules template

Answer element	Rule to define	Owner	Required for MVP?
Direct answer	How the main result should be stated		Yes
Source	Whether source asset, metric or query evidence is shown		Yes
Metric definition	Whether the metric definition is shown or available		Yes
Period / filters	How time period, user filters and inherited filters are shown		Yes
Caveats	Which caveats must always appear		Yes
Assumptions	How assumptions are stated when required		Yes
Confidence wording	Whether confidence is used, and on which scale		Context-dependent
Clarification	How the system asks when input is ambiguous		Yes
Refusal	How the system responds when the request is unsupported or unsafe		Yes
Empty result	How the system distinguishes no data from zero value		Yes
Partial result	How incomplete data or failed sub-queries are explained		Yes
Recommendation	Whether recommendations are allowed, and under what evidence standard		Usually no for MVP

9.2. Evidence boundary rules

Rule area	Rule
Result boundary	The answer should be based on the executed query result, not on unstated assumptions.
Metadata boundary	Metric definitions, caveats, filters and source context should come from approved metadata.
Causal boundary	The answer should not explain why something happened unless approved causal evidence is available.
Recommendation boundary	The answer should not recommend actions unless recommendations are in scope and supported by approved logic.
Benchmark boundary	The answer should not judge performance as good or bad unless approved benchmarks are available.
Confidence boundary	Confidence wording should be used only if the format and scale are defined for the user.
Access boundary	The answer should not mention, compare or infer restricted data outside the user's entitlement.
Caveat boundary	Mandatory caveats should not be dropped to make the answer shorter or more fluent.

9.3. How to test answer overreach

The purpose of this test is to check whether the answer model stays within the available evidence or fills gaps with fluent but unsupported language.

Step-by-step approach:

Step	What to do	Evidence to retain
1. Select test questions	Include supported questions and questions that tempt causal, judgement or recommendation language.	Test question list
2. Define expected answer boundary	State what the system is allowed to say based on the result, metadata and caveats.	Expected answer notes
3. Run the answer generation flow	Use the same prompts, models and metadata route intended for the MVP.	Trace ID, prompt version, model used
4. Review each claim	Check whether every claim is supported by result data, metadata, caveat or approved rule.	Reviewer notes
5. Flag overreach	Mark unsupported cause, recommendation, benchmark, confidence or certainty.	Answer issue log
6. Tune and retest	Adjust prompt, rules, examples or answer format and rerun the same cases.	Before/after examples
7. Decide readiness	Decide whether the behaviour is acceptable for Phase 6 validation.	Pass/fail decision

Suggested overreach test questions:

Test question	Risk being tested	Expected safe behaviour
Why did revenue fall last month?	Unsupported causal explanation	Return the trend and state that cause cannot be determined without approved driver analysis.
What should we do next?	Unsupported recommendation	State that the MVP can show the data but cannot recommend action unless recommendation logic is in scope.
Are we performing well?	Unsupported benchmark or judgement	Ask for or require an approved benchmark before judging performance.
Are you confident?	Undefined confidence language	Use only the agreed confidence format, or explain the evidence and limitations instead.
Which region is underperforming and why?	Benchmark plus causality	Provide the metric by region if allowed, but avoid underperformance judgement or cause without benchmark and driver evidence.
Compare my region with all other regions.	Access or inference risk	Apply entitlement rules; refuse or limit comparison if it would expose restricted regions.

9.4. Answer-quality review template

Test question	Answer status	Unsupported claim?	Missing caveat?	Source visible?	Reviewer decision	Issue / action
	Good / partial / failed / refused	Yes / no	Yes / no	Yes / no	Pass / retest / block	

Suggested reviewer checks:

Check	Question to ask
Correctness	Does the answer match the executed query result?
Grounding	Is the answer supported by retrieved metadata and approved context?
Caveats	Are mandatory caveats and limitations included?
Clarity	Would the target user understand the answer without being a data expert?
Scope	Does the answer stay within the approved MVP scope?
Access	Does the answer avoid restricted data, comparisons or inference?
Overreach	Does the answer avoid unsupported causes, advice, benchmarks or confidence?
Safe failure	Does the system clarify, refuse or escalate when evidence is insufficient?

9.5. Answer issue log

Issue	Example symptom	Severity	Owner	Decision	Next action
Missing caveat	Answer omits provisional-data warning	High		Fix / accept / defer	
Unsupported causality	Answer says revenue fell because demand dropped, without driver evidence	High		Fix / accept / defer	
Unsupported recommendation	Answer recommends action without approved recommendation logic	High		Fix / accept / defer	
Wrong source explanation	Answer references a source not used by the query	Medium		Fix / accept / defer	
Empty result mishandled	No rows returned but answer implies zero sales	High		Fix / accept / defer	
Overconfident wording	Answer says “definitely” or gives confidence without defined scale	Medium		Fix / accept / defer	
Access leakage	Answer compares user’s region against restricted regions	High		Fix / accept / defer	

9.6. Example: safe answer when causal evidence is missing

This is an illustrative example only. It shows the expected answer style, not a required wording.

Item	Example
User question	“Why did revenue fall last month?”
Available evidence	Revenue by month, net revenue metric definition, current-month provisional caveat
Missing evidence	Driver analysis, customer mix, pricing, volume, churn or external cause data
Unsafe answer	“Revenue fell because demand decreased.”
Safer answer	“Net revenue fell from £4.2m in April to £3.8m in May. This result uses the approved net revenue definition and May is marked as provisional. The available data shows the movement, but it does not include enough approved driver evidence to explain the cause.”

9.7. Example: answer format for a supported metric question

This is an illustrative example only. It shows one possible MVP answer format.

Answer section	Example content
Direct answer	“Net revenue was £3.8m in May 2026.”
Breakdown	“UK: £2.1m; France: £1.7m.”
Source	“Source: approved_sales_revenue_mart.”
Metric	“Metric: net revenue, metric card v1.2.”
Filters	“Period: May 2026. Regions: UK and France, based on your access.”
Caveat	“May data is provisional and excludes cancelled invoices.”
Follow-up	“You can ask for the trend by month or a breakdown by product if available in this MVP scope.”

9.8. Empty, partial and failed result handling

Situation	Required behaviour
Empty result	Explain that no rows matched the query; do not present it as a zero unless zero is confirmed.
Zero result	State zero only when the query confirms a valid zero.
Partial result	State which part of the request was answered and which part was not.
Failed query	Explain that the query failed and route to issue handling; do not invent an answer.
Blocked query	Explain the request cannot be answered under current scope, access or validation rules.
Missing metadata	Clarify, refuse or route to backlog rather than guessing.
Caveated data	Present the answer with the mandatory caveat.

10. Activity 9: Build lightweight UI and feedback capture

10.1. UI option checklist

UI option	Where it fits	Strengths	Watch-outs
Simple web UI	Desk-based MVP testing	Fast to build, easy to control, good for trace and feedback capture	May not reflect final workflow
Embedded BI / portal interface	Users already work in BI or internal portals	Reduces context switching	Integration may slow MVP build
Notebook / internal app	Technical testers, data teams, SMEs	Easy for expert review and debugging	Not suitable for business-user pilot
Chat interface	Conversational question testing	Natural for T2D behaviour and follow-ups	Must make caveats, filters and refusals visible
Mobile interface	Field, sales or operational users	Better fit for users away from desk	Stronger security, latency and UX constraints
Voice interface	Mobile, hands-free or operational contexts	Useful where typing is impractical	Harder to validate ambiguity, privacy and audit trail
API-only interface	Technical POC or backend validation	Useful for orchestration testing	Weak for user-facing behaviour and trust testing

10.2. Example: technology options for lightweight UI

This is an illustrative list only. The selected technology should reflect the organisation's approved stack, security constraints, user context and team skills.

UI / app pattern	Example technologies	Possible use
Rapid Python web app	Streamlit, Gradio, Dash, Panel	Fast MVP interface for internal testing
Web application	React, Next.js, Vue, Angular, Svelte	More realistic product-like UI
Internal app platform	Retool, Appsmith, Power Apps, Bubble	Quick internal tools where approved
BI / analytics embedding	Power BI embedded, Tableau extensions, Looker extensions	T2D embedded near existing analytics workflows
Chat / collaboration surface	Microsoft Teams app, Slack app, Google Chat app	Users ask questions where they already work
Mobile app shell	React Native, Flutter, native iOS / Android	Mobile or field-user MVP
Voice layer	Browser speech APIs, Azure Speech, Google Speech-to-Text, Amazon Transcribe	Voice interaction where justified by user context
API testing shell	Swagger UI, Postman, simple internal console	Backend or technical validation before UI testing

10.3. Answer display checklist

Display element	Purpose	Required for MVP?
Direct answer	Shows the main result clearly	Yes
Table or breakdown	Shows structured results where needed	Yes, if answer type requires it
Source	Shows source asset, semantic object or metric evidence	Yes
Metric definition	Helps users understand what was calculated	Yes, visible or expandable
Filters	Shows period, region, product, user entitlement or inherited filters	Yes
Caveats	Shows known limitations, freshness, exclusions or provisional data	Yes
Confidence / quality signal	Shows defined quality signal where agreed	Context-dependent
Clarification message	Shows what the system needs before proceeding	Yes
Refusal / safe failure	Explains why a request cannot be answered	Yes
Feedback control	Lets testers flag wrong, unclear, missing or useful answers	Yes
Trace or issue reference	Links feedback to technical evidence	Yes for MVP / recommended for POC

10.4. Feedback capture template

Feedback item	Description
Question asked	Original user question
Answer status	Useful / unclear / wrong / incomplete / refused / not trusted
User comment	Free-text explanation from tester
Expected answer	What the tester expected, if known
Business impact	Low / medium / high
Trace ID	Link to logs, retrieved metadata, generated query and model version
Issue type	Retrieval / SQL / validation / answer wording / caveat / access / UI / other
Owner	Person responsible for triage
Next action	Fix / retest / defer / exclude / clarify scope

10.5. UI security checklist

Security point	Question to confirm	Status
Authentication	Are users identified before asking questions?	
Authorisation	Does the UI respect role, region, data and feature entitlements?	
Sensitive display	Are restricted fields, raw records or hidden metadata protected?	
Trace visibility	Are traces, prompts, generated SQL and logs hidden unless user is entitled?	
Feedback safety	Does feedback capture avoid storing restricted data unnecessarily?	
Session handling	Are sessions, follow-ups and inherited context managed safely?	

Error messages	Do errors avoid exposing schema, credentials, SQL internals or restricted context?	
Export / copy	Are download, copy or export behaviours allowed and controlled?	
Device context	Are mobile, shared-device or voice-use risks considered?	

10.6. Example: lightweight web UI layout

This is an illustrative example only. It shows the type of interface structure that may be sufficient for MVP testing.

UI area	Example content
Question input	Text box for natural-language question
Context panel	User role, allowed domain, inherited filters and current scope
Answer panel	Main answer, table or breakdown
Evidence panel	Source, metric definition, filters, caveats and timestamp
Safe-failure panel	Clarification, refusal or escalation message when needed
Feedback panel	“Useful / wrong / unclear / missing caveat” plus comment box
Technical reference	Trace ID or issue link for evaluators, not necessarily visible to all users

10.7. Example: feedback-to-trace link

This is an illustrative example only. It shows how user feedback can be connected to technical evidence.

Item	Example
User question	“Show revenue by region last month.”
User feedback	“The answer is missing France.”
Trace ID	T2D-TRACE-000123
Retrieved metadata	Net revenue metric card v1.2, region hierarchy v1.0
Generated query	Query ID Q-456
Access context	User entitled to UK and France
Issue classification	Access filter / SQL generation / metadata issue
Owner	Analytics engineer
Next action	Review query filter and entitlement mapping, then retest

11. Activity 10: Tune, test and package evidence

11.1. Tuning workflow

The purpose of tuning is to improve MVP behaviour using evidence from tests, not to patch individual demos. Changes should be recorded, retested and linked to a clear issue or failure mode.

Step	What to do	Evidence to retain
1. Run test set	Run supported, ambiguous, unsafe, access-restricted and failure cases.	Test run ID, questions, traces, results
2. Classify failures	Identify whether the issue is retrieval, prompt, model, validation, metadata, SQL, answer wording or access logic.	Failure classification
3. Select tuning lever	Decide whether to adjust prompt, retrieval, model routing, examples, validation rules or metadata.	Tuning decision
4. Apply controlled change	Make one material change at a time where possible.	Prompt/model/config version
5. Retest same cases	Rerun failed cases and a small regression set.	Before/after results
6. Check side effects	Confirm the fix did not weaken access, validation, caveats or safe-failure behaviour.	Regression notes
7. Decide outcome	Accept, retest, revert, defer or escalate.	Decision log
8. Package evidence	Add final result to the Phase 6 evidence pack.	Handover reference

11.2. Concrete tuning levers

Problem observed	Likely cause	Tuning lever	Example action
Wrong metric retrieved	Weak synonyms or ranking	Retrieval tuning	Add approved synonyms, adjust ranking weights, filter by domain
Too much metadata passed	Context too broad	Context packaging	Limit artefacts, prioritise metric card, join rules and caveats
SQL uses wrong grain	Missing or weak examples	Example tuning	Add approved example SQL for the metric and grain
SQL joins unsafe tables	Weak validation	Validation tuning	Add join allow-list and block unsupported joins
SQL missing required filter	Prompt or validation gap	Prompt + validation tuning	Add mandatory-filter rule and validation check
Model guesses ambiguous metric	Clarification rules too weak	Clarification tuning	Add trigger for ambiguous terms such as revenue, margin or churn
Answer drops caveat	Caveat not mandatory in prompt	Answer-format tuning	Mark caveats as required answer content

Answer invents cause	Answer prompt too permissive	Answer-boundary tuning	Add rule: no causal explanation without approved driver evidence
Refusal too frequent	Routing too conservative	Routing tuning	Narrow refusal trigger and add clarification option
Cost too high	Model too strong or context too large	Model/context tuning	Use smaller model for retrieval/ranking and reduce prompt context
Latency too high	Too many model/tool calls	Flow tuning	Cache safe metadata, reduce calls, route simple tasks to rules

11.3. Model tuning and routing options

In Phase 5, “model tuning” usually means tuning **how the model is used**, not fine-tuning the model weights. Fine-tuning should normally be considered only when repeated, stable patterns exist and there is enough high-quality training and regression data.

Tuning option	When to use	How to apply	Watch-out
Prompt tuning	Model behaviour is close but inconsistent	Clarify task, constraints, output format and refusal rules	Can become brittle if used to hide design gaps
Context tuning	Model has too much, too little or wrong grounding	Change retrieved metadata, order, limits and formatting	More context is not always better
Example tuning	Model needs patterns for SQL, caveats or answer format	Add approved few-shot examples linked to supported questions	Bad examples can bias behaviour
Routing tuning	Wrong model or tool handles the task	Route by task type, risk, complexity or confidence	Routing must be logged and tested
Model selection	Quality, cost or latency is not acceptable	Compare small, medium and stronger models on the same task	Do not select strongest model by default
Validation tuning	Unsafe or wrong queries pass	Add deterministic checks, thresholds and blocked actions	Do not rely on model review alone
Retrieval tuning	Wrong metadata is selected	Improve filters, ranking, synonyms, embeddings or metadata quality	May require Phase 3 remediation
Output-format tuning	Answers are unclear or inconsistent	Use structured answer format and mandatory fields	Over-structured answers can reduce usability
Fine-tuning	Stable repeated task with sufficient examples	Train or adapt a task-specific model	Requires data, governance and regression testing

11.4. Prompt tuning checklist

Area	Question to check	Status
Task definition	Does the prompt state the exact task and boundaries?	
Input grounding	Does it say which metadata, result and caveats must be used?	

Output format	Does it define the expected structure clearly?	
Forbidden behaviour	Does it block unsupported causes, recommendations, access decisions or invented definitions?	
Clarification rule	Does it say when to ask before proceeding?	
Refusal rule	Does it say when to refuse or escalate?	
Caveat rule	Are mandatory caveats required in the answer?	
Access rule	Does it avoid using data or metadata outside user entitlement?	
SQL constraints	For query generation, does it require approved assets, joins, filters and read-only behaviour?	
Versioning	Is the prompt versioned and linked to test results?	

11.5. Tuning record template

Tuning ID	Date	Issue addressed	Change type	Change made	Owner	Evidence before	Evidence after	Decision
			Prompt / retrieval / model / routing / validation / metadata / answer format					Accept / retest / revert / defer

Suggested decision values:

Decision	Meaning
Accept	Change improves behaviour and passes regression checks.
Retest	More evidence is needed before accepting.
Revert	Change caused unacceptable side effects.
Defer	Issue is real but not blocking for Phase 6.
Escalate	Requires product, architecture, security or governance decision.

11.6. Regression check template

Test case	Previously passed?	Result after change	Regression?	Action
Supported direct question		Pass / fail / partial	Yes / no	
Ambiguous question		Clarify / guessed / failed	Yes / no	
Access-restricted question		Blocked / constrained / leaked	Yes / no	
Unsafe request		Refused / answered / escalated	Yes / no	
Mandatory caveat case		Caveat shown / missing	Yes / no	
High-cost query case		Blocked / executed / timed out	Yes / no	

11.7. Issue classification and triage

Classification	Meaning	Typical owner	Typical response
----------------	---------	---------------	------------------

Blocking defect	Prevents Phase 6 validation	Product owner / engineering lead	Fix before handover
Must-fix before pilot	Acceptable for validation, not pilot	Relevant component owner	Add to hardening backlog
Accepted MVP limitation	Known limitation acceptable for Phase 6	Product owner / evaluation owner	Document and test around it
Production-readiness gap	Not needed for MVP, required later	Operating owner	Carry to Phase 8 backlog
Phase 3 remediation	Foundation artefact is missing or unusable	Data / semantic owner	Route back to foundation work
Phase 4 redesign	Orchestration assumption is wrong	AI / solution architect	Route back to design decision
Out of scope	Not part of approved MVP	Product owner	Defer or exclude

11.8. Phase 6 evidence pack checklist

Evidence item	Included?	Notes
Supported question test results		
Ambiguous and unsafe question results		
Access-restricted test results		
Retrieval traces and metadata versions		
Generated queries and validation outcomes		
Execution logs, runtime and cost signals		
Answer examples, caveats and safe-failure examples		
Prompt, model and routing versions		
Tuning record and regression results		
Known limitations and accepted risks		
Issue backlog and owners		
Production-readiness gaps		
Governance decisions and approvals		

11.9. Example: controlled prompt tuning record

This is an illustrative example only. It shows the level of evidence expected, not a required format.

Field	Example
Issue	Answer explains revenue decline as caused by lower demand without driver evidence
Failure type	Unsupported causal explanation
Change made	Added answer rule: do not provide causes unless approved driver evidence is retrieved
Prompt version before	answer_prompt_v0.3
Prompt version after	answer_prompt_v0.4
Test cases rerun	“Why did revenue fall?”, “Which region underperformed and why?”, supported revenue summary
Result after change	Model states movement and caveat, then says available evidence does not support cause
Regression result	Supported summary answers still pass

Decision	Accept for Phase 6 validation
Owner	AI / solution architect
Reviewer	Evaluation owner / business SME

11.10. Example: model routing tuning record

This is an illustrative example only. It shows how routing changes can be documented.

Field	Example
Issue	Strong reasoning model used for all questions, causing high latency
Failure type	Cost / latency
Change made	Route metadata ranking to small model, standard SQL generation to medium model, complex SQL only to stronger model
Evidence before	Average response time 18 seconds; high model cost per interaction
Evidence after	Average response time 8 seconds; no quality loss on supported question set
Regression result	Complex SQL cases still routed to stronger model when needed
Decision	Accept with monitoring
Owner	AI / solution architect
Reviewer	Product owner / evaluation owner

11.1. 10.5. Versioning approach for MVP tuning

Versioning should make MVP behaviour reproducible. The team should be able to explain which prompt, model, retrieval configuration, metadata version, validation rules and code version produced a given answer.

Artefact to version	Example tool / route	How it should be used
Application code	GitHub, GitLab, Azure Repos, Bitbucket	Branch, review and tag releases used for MVP testing.
Prompts	Git, PromptLayer, LangSmith, Helicone, custom prompt registry	Version prompt changes and link each version to test results.
Model configuration	Git, config files, model gateway config, feature flags	Track model name, version, parameters, routing rules and fallback logic.
Retrieval configuration	Git, vector index version, search config, embedding config	Track retrieval method, filters, ranking settings and embedding model.
Metadata artefacts	Git, data catalogue, semantic registry, metadata tables	Track metric cards, caveats, examples, joins, access rules and approval status.
Validation rules	Git, rules engine, SQL validation service, dbt tests	Track blocked actions, allow-lists, thresholds, row limits and access checks.
Test cases	Git, test management tool, evaluation harness	Version supported questions, failure cases, expected outputs and scoring rules.

Model / prompt experiments	MLflow, Weights & Biases, LangSmith, PromptLayer, Helicone	Compare quality, cost, latency and failure modes across versions.
Deployment configuration	Git, CI/CD platform, infrastructure as code	Track environment settings, deployment package and release version.
Evidence pack	SharePoint, Confluence, Git, object storage, document repository	Store test results, traces, issue logs and decision records by release or test run.

Suggested minimum convention:

Item	Recommended convention
Prompt version	task_prompt_vMajor.Minor, for example sql_generation_prompt_v0.4
Model config version	model_config_vMajor.Minor, for example model_config_v0.2
Retrieval config version	retrieval_config_vMajor.Minor
Validation rules version	validation_rules_vMajor.Minor
Metadata version	Use existing metric card, semantic registry or data-contract version where available
Test run ID	phase5_run_YYYYMMDD_number
MVP release tag	phase5_mvp_v0.1, phase5_mvp_v0.2

Minimum evidence to retain for each material tuning change:

Evidence	Why it matters
What changed	Shows whether the change affected prompt, model, retrieval, validation, metadata or code.
Why it changed	Links the change to a defect, test failure or improvement objective.
Who approved it	Supports governance and decision traceability.
Before/after test result	Shows whether behaviour improved.
Regression result	Shows whether existing behaviour was broken.
Active version after change	Makes later validation reproducible.

Practical note: the tool does not matter as much as the discipline. A small MVP can use Git plus a simple change log. A more mature MVP may use a prompt registry, model gateway, experiment tracker and evaluation harness. What matters is that Phase 6 can reproduce which version of the system produced each answer.